Week 7: Cross-Site Request Forgery

Learning Objectives

By the end of this lecture, the student should be able to:

- *Define Cross-Site Request Forgery and distinguish it from Cross-Site Scripting, examined in Week 6.*
- *Explain the browser behavior that Cross-Site Request Forgery depends on, and describe this behavior formally as an instance of ambient authority and the confused deputy problem.*
- *Explain why a state changing action exposed through a GET request increases exposure to this attack.*
- *Describe the Synchronizer Token Pattern and the double submit cookie pattern, including why current guidance favors a signed, session bound variant over the naive version.*
- *Compare the reliability of the Origin and Referer headers when verifying the declared source of a request.*
- *Explain how the SameSite cookie attribute, introduced in Week 2, reduces exposure by default in modern browsers, and identify the specific cases, including a clicked link and a temporary browser compatibility exception, where that default still permits a forged request.*
- *Explain why a Cross-Site Scripting vulnerability can defeat a token-based Cross-Site Request Forgery defense.*
- *Locate Cross-Site Request Forgery within the OWASP Top Ten and explain why it is no longer treated as a standalone category.*

Cross-Site Request Forgery as a Failure of Ambient Authority

The mechanics above are best understood through a formal concept from security theory. A subject is said to exercise ambient authority when it can perform an action on a resource merely by naming that resource, without separately presenting a specific permission scoped to that particular action. Browser cookies behave exactly this way: the browser attaches a stored cookie to a request based only on the destination domain, never checking which page or code actually initiated the request or why. This design is convenient for ordinary browsing, but it cannot distinguish an action the user meant to take from an action an unrelated page silently triggered on the user's behalf.

Security literature describes this general pattern as the confused deputy problem: a program holding more authority than the party asking it to act is tricked into misusing that authority on the requester's behalf. In this scenario, the browser is the deputy, the session cookie is the authority, and the malicious page is the party that tricks the deputy into spending an authority it was never given directly. Framing the attack this way explains why token-based defenses work in principle: a token replaces implicit, ambient

authorization with an explicit, per action permission that only code running on the legitimate origin can obtain.

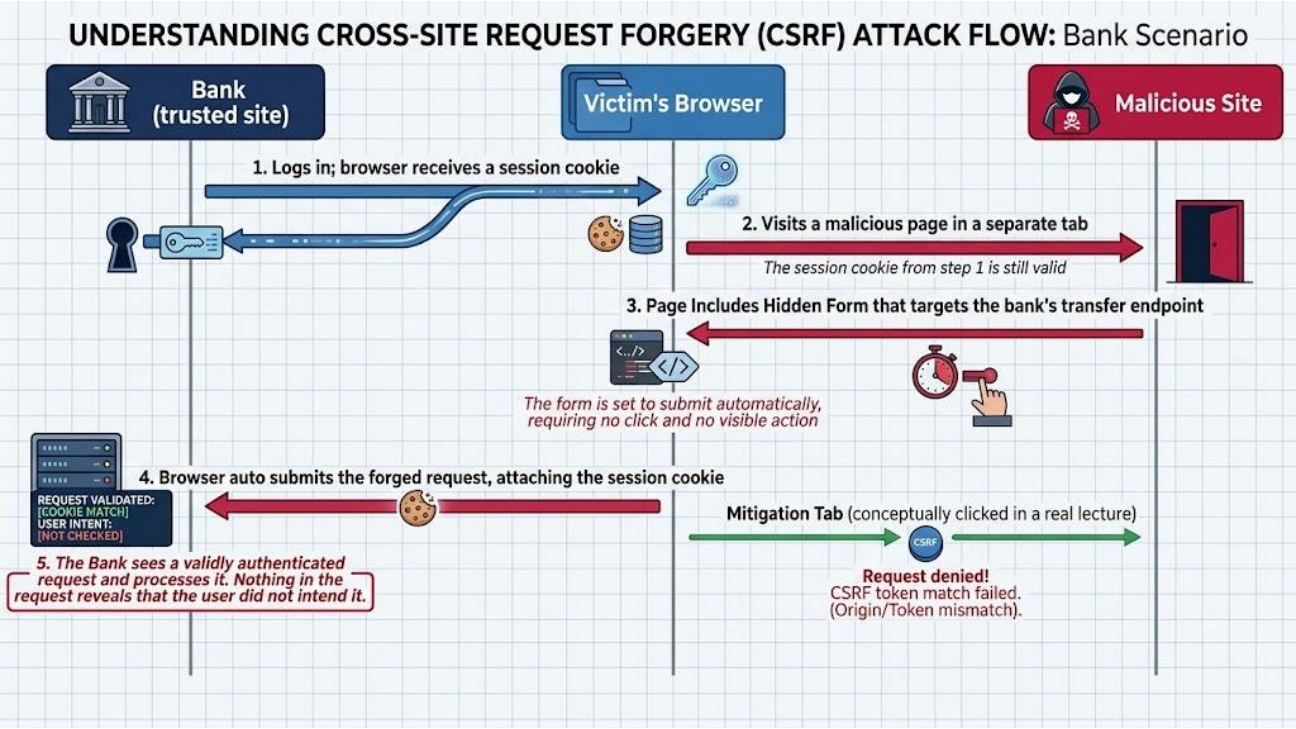


Figure 1. The victim never leaves an authenticated session; the malicious page merely causes the browser to spend that authentication on a request the victim never intended.

Risks of State-Changing GET Requests

Week 2 established that GET requests are expected to be safe, meaning they should not alter server-side state. Cross-Site Request Forgery is one of the clearest illustrations of why that expectation matters in practice. A forged GET request needs nothing more elaborate than an ordinary image tag or a link pointing at the target endpoint; the browser issues the request the moment the tag is parsed, with no script and no user interaction required. A forged POST request at least requires an automatically submitting form, a small extra step for the attacker but still trivial to construct. An application that exposes a state changing action, such as a funds transfer or a password change, through a GET request has removed even that minor obstacle.

Primary Defenses

No single defense below is complete on its own. Effective protection typically layers a token-based defense with the ***SameSite*** cookie attribute, treating the cookie attribute as a broad safety net and the token as the mechanism that holds even when the safety net does not apply.

Defense	Mechanism	Primary limitation
Synchronizer Token Pattern	The server generates a unique, unpredictable token per session, embeds it in every state changing form, and rejects any submission whose token does not match.	Requires server-side state and must be included in every form or API call.
Double submit cookie	A random value is set as a cookie and must also be repeated as a request parameter. An attacker on another origin cannot read the cookie in order to copy the value.	The plain version is weaker unless the value is signed and bound to the session, as recommended in current guidance.
Custom request header	Client-side script adds a custom header to state changing requests. Only script running on the same origin is permitted to set it.	Not usable for a plain form submission or a followed link, only for requests made through script.
SameSite cookie attribute	Instructs the browser to withhold the cookie on most cross-site requests, removing the ambient authority the attack depends on.	The Lax setting, the modern default, still allows the cookie on a top-level navigation such as a clicked link.
Origin or *Referer* verification	The server compares the declared source of the request against its own origin and rejects a mismatch.	The *Referer* header can be withheld by a strict Referrer Policy, covered in Week 2, leaving nothing to compare.



Figure 2. A valid session cookie is necessary but no longer sufficient once the endpoint also requires a token the forged request cannot supply.

Strengthening the Double Submit Cookie Pattern

The double submit cookie pattern described above is often called the naive variant, and it assumes an attacker cannot set cookies on the target domain. That assumption does not always hold. A vulnerable subdomain, an abandoned DNS record that enables a subdomain takeover, or a connection still reachable over plain HTTP can each allow an attacker to write a cookie that matches the value the application expects, defeating the comparison entirely. Current OWASP guidance therefore recommends a signed double submit cookie instead, in which the value is generated with a server-side secret through an HMAC construction and explicitly bound to the user's authenticated session identifier. An attacker who manages to set an arbitrary cookie still cannot produce a value that passes the signature check without knowing the server's secret key, closing the gap the naive version leaves open.

Comparing the Origin and *Referer* Headers

The Origin and ***Referer*** headers serve a similar verification purpose but differ in reliability. The Origin header is included in every cross-origin request and, in current browsers, in same origin POST, PUT, and DELETE requests as well; it reports only the scheme, host, and port of the requesting page, never the path, which keeps it compact and avoids leaking browsing details unnecessarily. The ***Referer*** header, by contrast, carries the full URL of the requesting page and can be suppressed entirely by a strict Referrer Policy, the header introduced in Week 2, or withheld by privacy focused browser settings. For this reason, current guidance treats the Origin header as the more dependable signal when it is present, falling back to ***Referer*** only when Origin is absent.

Limitations of the *SameSite* Default in Practice

The ***SameSite*** attribute defaulting to Lax, in effect in major browsers since 2020, blocks the cookie on most cross-site requests but was deliberately designed to still permit it on a top-level navigation performed through a GET request, such as a link the victim clicks, since blocking that case would break ordinary cross site linking across the web. A forged GET based request therefore still succeeds under the default setting, which is exactly why the earlier discussion of state changing GET requests in this lecture matters in practice and not only in principle. Chromium based browsers additionally ship a temporary compatibility exception that allows a cookie no more than two minutes old to accompany a top-level cross site POST request, added to avoid breaking certain identity federation and single sign on flows that depend on a cross-site POST shortly after a cookie is set. Browser vendors have described this exception as temporary and expected to be removed. None of these details change the central guidance of this lecture: the ***SameSite*** attribute is a valuable default, not a substitute for an explicit token.

Interaction Between XSS and Token-Based Defenses

A Cross-Site Request Forgery token assumes that the attacker's page cannot read anything belonging to the target site. A Cross-Site Scripting vulnerability on that same site breaks this assumption directly, since injected script runs with the full privileges of the page, including the ability to read a token embedded in the page or stored in a cookie the script can access. Once the token can be read, it can simply be copied into the forged request, and the defense examined in this lecture no longer holds. This is the clearest practical link between the two lectures: a Cross-Site Scripting flaw does not just endanger the page it appears on; it can quietly remove the protection the application was relying on against an entirely different attack.

Placement Within the OWASP Top Ten

Cross-Site Request Forgery ranked fifth in the 2007 edition of the OWASP Top Ten and eighth in the 2013 edition, then dropped out of the list entirely starting with the 2017 edition. OWASP cited a declining incidence rate together with the fact that most modern web frameworks had begun including built in protections against it. The 2021 edition, used as the reference throughout this course, does not list Cross-Site Request Forgery as its own category, but the underlying weakness, catalogued as CWE-352, is explicitly included under A01:2021, Broken Access Control. That placement reflects the nature of the flaw accurately: an application that processes a state changing request without verifying that the request reflects the genuine intent of an authorized user has failed to control access to that action, regardless of whether the session cookie presented was valid.